

Ứng dụng cấu trúc cây trong lập trình

Zhou Wenchao

Nguồn gốc: Applications of tree structures in programming

IOI China candidate team papers / 2002 / Zhou Wenchao.pdf

Mục lục

1	Dẫn nhập	1
2	Tập hợp rời nhau	2
2.1	Cách lưu bằng danh sách liên kết	2
2.2	Cách lưu bằng cây	2
2.3	Nén đường đi	3
3	Cây đoạn	3
3.1	Định nghĩa	3
3.2	Thêm và xóa đoạn	4
3.3	Độ đo	4
3.4	Số đoạn liên tục	4
4	Mảng cây	5
4.1	Từ tổng tiền tố một chiều	5
4.2	Xây dựng mảng C	5
4.3	Cập nhật giá trị phần tử	6
4.4	Tính tổng dãy con	7
4.5	Tổng ma trận con	8
5	Kết luận	9

1 Dẫn nhập

Trong những năm gần đây, vì nhiều kỳ thi lập trình đã chuyển sang dùng Free Pascal, yêu cầu về hiệu quả bộ nhớ đối với thuật toán được nói lỏng hơn, trong khi yêu cầu về thời gian chạy lại cao hơn. Điều đó đòi hỏi thí sinh không chỉ nắm chắc các thuật toán thông thường, mà còn phải mạnh dạn cải tiến và xây dựng thuật toán hiệu quả hơn để giải quyết bài toán.

Trong lập trình trước đây, cấu trúc liên kết được dùng khá nhiều. Cấu trúc liên kết thật sự có ưu điểm là dễ lập trình và dễ hiểu, nhưng nó có một điểm yếu chí mạng: quan hệ giữa hai phần tử kề nhau không

được thể hiện rõ ràng. Cấu trúc cây lại làm được điều này rất tốt. Bài viết giới thiệu ba ví dụ tiêu biểu: tập hợp rời nhau, cây đoạn và mảng cây.

2 Tập hợp rời nhau

Trong thi đấu, ta thường gặp dạng bài: cho các quan hệ giữa các phần tử, hãy chia các phần tử thành một số tập sao cho các phần tử trong cùng một tập có liên hệ trực tiếp hoặc gián tiếp với nhau. Dạng bài này chủ yếu cần hai thao tác: hợp nhất hai tập và tìm tập chứa một phần tử. Vì vậy cấu trúc dữ liệu tương ứng thường được gọi là **tập hợp rời nhau**, hay **union-find**.

2.1 Cách lưu bằng danh sách liên kết

Danh sách liên kết từng được dùng phổ biến để hiện thực union-find. Mỗi phần tử trong danh sách có hai con trỏ: một con trỏ trỏ tới phần tử kế tiếp trong cùng tập, con trỏ còn lại trỏ tới phần tử đầu danh sách. Với cách lưu này, thao tác tìm tập có độ phức tạp chỉ $O(1)$, vì từ một phần tử có thể đến ngay đại diện của tập. Tuy nhiên, thao tác hợp nhất hai tập có thể phải đổi con trỏ đầu danh sách của mọi phần tử trong một tập, nên độ phức tạp lên đến $O(n)$. Nếu muốn cả hai thao tác cơ bản đều nhanh, cấu trúc liên kết không còn phù hợp.

2.2 Cách lưu bằng cây

Cấu trúc cây có thể hỗ trợ union-find tốt hơn. Khác với cây thông thường, mỗi đỉnh không ghi danh sách các con mà ghi cha của nó. Gốc của cây chính là đại diện của tập. Với mảng `parent`, ta có:

$$\text{parent}[x] = x \iff x \text{ là gốc của cây.}$$

Hai thao tác cơ bản có thể viết như sau.

```
function FIND(x)
begin
    while parent[x] <> x do x <- parent[x]
    return x
end

procedure UNION(x, y)
begin
    rx <- FIND(x)
    ry <- FIND(y)
    if rx <> ry then parent[rx] <- ry
end
```

Việc hợp nhất hai tập rất đơn giản: chỉ cần nối gốc của một cây vào gốc của cây kia, thao tác nối này mất $O(1)$ sau khi đã tìm được hai gốc. Do đó thời gian của union-find phụ thuộc chủ yếu vào tốc độ tìm gốc. Nếu truy vấn trực tiếp trong cây, thời gian tìm tuyến tính theo độ sâu của cây. Trung bình có thể xem là $O(\log N)$ khi cây khá cân bằng, nhưng trong trường hợp xấu nhất cây suy biến thành một dây, làm mỗi lần tìm tốn $O(N)$.

2.3 Nén đường đi

Ta còn có thể giảm độ phức tạp của thao tác tìm bằng **nén đường đi**. Ý tưởng rất đơn giản: trong lúc tìm gốc của một phần tử, đồng thời làm giảm độ sâu của các đỉnh nằm trên đường đi đó bằng cách cho chúng trở thẳng tới gốc.

```
function FIND(x)
begin
  if parent[x] <> x then parent[x] <- FIND(parent[x])
  return parent[x]
end
```

Sau khi dùng nén đường đi, mỗi truy vấn có độ phức tạp khấu hao là hàm ngược của hàm Ackermann, ký hiệu $\alpha(n)$, một hàm tăng cực kỳ chậm. Với mọi giá trị n có thể gặp trong thực tế, $\alpha(n)$ không vượt quá 5.

So với cách lưu bằng danh sách liên kết, cách lưu bằng cây có ưu thế rất rõ: cài đặt vẫn ngắn gọn, trong khi hiệu quả thời gian cao hơn nhiều.

3 Cây đoạn

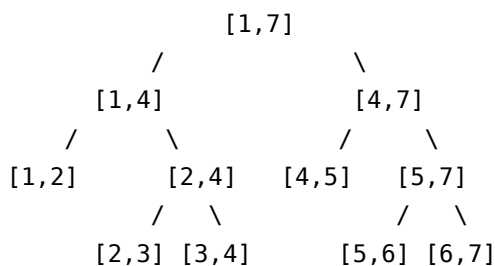
Khi xử lý các bài toán liên quan đến diện tích, chu vi và các đại lượng hình học tương tự, ta thường không cần kiến thức toán học quá sâu; khó khăn nằm ở việc nâng cao hiệu quả xử lý. Muốn vậy, nhiều khi phải thay đổi nền tảng của thuật toán, tức thay đổi cấu trúc dữ liệu. Cây đoạn là một cấu trúc dữ liệu đặc biệt phục vụ mục tiêu đó.

Ta xét bài toán cơ sở: cho n đoạn thẳng trên một trục, hãy xác định độ dài phần được các đoạn này phủ. Qua bài toán này, ta lần lượt thấy định nghĩa cây đoạn, thao tác thêm/xóa đoạn, và cách tính độ đo cùng số đoạn liên tục.

3.1 Định nghĩa

Cây đoạn là một cây nhị phân chia một khoảng thành các khoảng đơn vị $[i, i + 1]$. Mỗi khoảng đơn vị tương ứng với một lá trong cây. Mỗi nút biểu diễn một khoảng $[l, r]$, và có một biến count ghi số đoạn đang phủ hoàn toàn khoảng của nút đó.

Ví dụ, cây đoạn của khoảng $[1, 7]$ có thể chia như sau. Trong ví dụ có một đoạn $[3, 6]$; các khoảng sơ cấp bị đoạn này phủ là $[3, 4]$, $[4, 5]$ và $[5, 6]$.



Đoạn $[3, 6]$ phủ các lá $[3, 4]$, $[4, 5]$ và $[5, 6]$.

3.2 Thêm và xóa đoạn

Thao tác thêm và xóa đoạn trong cây đoạn giống nhau về cấu trúc. Ta quét đệ quy xuống các con cho đến khi đoạn cần xử lý phủ hoàn toàn khoảng của nút hiện tại; khi đó chỉ cập nhật count tại nút này.

```
procedure ADD(v, L, R, delta)
begin
  if L <= left[v] and right[v] <= R then
    count[v] <- count[v] + delta
  else
    begin
      mid <- (left[v] + right[v]) div 2
      if L < mid then ADD(leftChild[v], L, R, delta)
      if mid < R then ADD(rightChild[v], L, R, delta)
      PULL(v)
    end
  end
end
```

Khi $\delta = 1$, thủ tục là thêm đoạn; khi $\delta = -1$, thủ tục là xóa đoạn. Phân tích cho thấy thời gian thêm hoặc xóa một đoạn đều là $O(\log N)$, với N là số khoảng đơn vị hoặc số điểm chia sau khi rời rạc hóa.

3.3 Độ đo

Độ đo m của một nút là tổng độ dài trong khoảng của nút đang được phủ bởi ít nhất một đoạn. Nếu nút biểu diễn khoảng $[i, j]$, ta có:

$$m = \begin{cases} j - i, & \text{nếu count} > 0, \\ 0, & \text{nếu count} = 0 \text{ và nút là lá,} \\ m_{\text{left}} + m_{\text{right}}, & \text{nếu count} = 0 \text{ và nút là nút trong.} \end{cases}$$

Nói cách khác, nếu chính khoảng của nút đã bị một đoạn phủ hoàn toàn, độ đo của nút là toàn bộ chiều dài khoảng. Nếu không, độ đo phải được lấy từ hai cây con.

3.4 Số đoạn liên tục

Số đoạn liên tục, ký hiệu line , là số thành phần liên thông trong hợp các đoạn nằm trong khoảng của nút. Đại lượng này không thể tính bằng cách cộng đơn giản số đoạn liên tục của hai con, vì một đoạn ở con trái và một đoạn ở con phải có thể nối liền nhau tại ranh giới.

Do đó ta thêm hai biến. Biến biên trái là

$$\text{lbd} = \begin{cases} 1, & \text{nếu đầu trái của khoảng được phủ,} \\ 0, & \text{nếu đầu trái của khoảng không được phủ.} \end{cases}$$

Biến biên phải là

$$\text{rbd} = \begin{cases} 1, & \text{nếu đầu phải của khoảng được phủ,} \\ 0, & \text{nếu đầu phải của khoảng không được phủ.} \end{cases}$$

Khi nút biểu diễn khoảng $[i, j]$, ta cập nhật line như sau:

$$\text{line} = \begin{cases} 1, & \text{nếu count} > 0, \\ 0, & \text{nếu count} = 0 \text{ và nút là lá,} \\ \text{line}_L + \text{line}_R - 1, & \text{nếu count} = 0, \text{ nút là nút trong,} \\ & \text{rbd}_L = 1 \text{ và lbd}_R = 1, \\ \text{line}_L + \text{line}_R, & \text{trong trường hợp còn lại.} \end{cases}$$

Trường hợp trừ 1 là trường hợp hai thành phần ở hai cây con chạm nhau tại điểm chia, vì đầu phải của con trái và đầu trái của con phải đều đang được phủ. Các biến biên cũng được cập nhật theo cùng nguyên tắc: nếu $\text{count} > 0$, cả hai biên đều bằng 1; nếu không thì lấy biên ngoài tương ứng từ hai con.

4 Mảng cây

Bài *Mobile* của IOI 2001 đã gây khó khăn cho nhiều thí sinh. Nội dung bài toán rất đơn giản: trong một ma trận, ta thay đổi giá trị phần tử để cập nhật trạng thái ma trận, đồng thời cần tính tổng các phần tử trong một ma trận con. Điểm khó nằm ở quy mô dữ liệu rất lớn. Phần này giới thiệu một cấu trúc dữ liệu mới: **mảng cây**, tức **Fenwick tree**.

Nhờ mảng cây, độ phức tạp lập trình tăng lên một chút, nhưng hiệu quả thời gian của chương trình tăng đáng kể. Tư tưởng chính vẫn là lợi dụng cấu trúc cây để thu hẹp phạm vi tìm kiếm và gom thông tin, làm cho mỗi lần cập nhật hoặc tính tổng chỉ liên quan đến ít biến nhất có thể.

4.1 Từ tổng tiền tố một chiều

Trước hết đơn giản hóa bài toán về một chiều. Cho dãy

$$a[1], a[2], \dots, a[n].$$

Cách thứ nhất là tính trực tiếp trên dãy gốc. Cập nhật một phần tử mất $O(1)$, nhưng trong trường hợp xấu nhất, tính tổng một dãy con mất $O(n)$.

Cách thứ hai là thêm mảng tiền tố b , trong đó

$$b[i] = a[1] + a[2] + \dots + a[i].$$

Khi đó truy vấn tổng dãy con có thể thực hiện trong $O(1)$, nhưng thay đổi $a[i]$ sẽ ảnh hưởng đến $b[i], b[i+1], \dots, b[n]$, nên cập nhật có thể tốn $O(n)$.

Hai cách trên đều làm một trong hai thao tác bị chậm. Ta cần một cách cân bằng hơn.

4.2 Xây dựng mảng C

Ta thêm mảng C , trong đó

$$C[i] = a[i - 2^k + 1] + a[i - 2^k + 2] + \dots + a[i],$$

với k là số chữ số 0 liên tiếp ở cuối biểu diễn nhị phân của i . Tương đương, $2^k = \text{lowbit}(i)$, nên $C[i]$ lưu tổng $\text{lowbit}(i)$ phần tử kết thúc tại vị trí i .

Từ định nghĩa của mảng C , ta có:

$$\begin{aligned}C[1] &= a[1], \\C[2] &= a[1] + a[2] = C[1] + a[2], \\C[3] &= a[3], \\C[4] &= a[1] + a[2] + a[3] + a[4] = C[2] + C[3] + a[4], \\C[5] &= a[5], \\C[6] &= a[5] + a[6] = C[5] + a[6].\end{aligned}$$

Cấu trúc bao phủ của mảng C tương ứng với một cây, nên được gọi là mảng cây. Sau khi phân tích hai thao tác cập nhật và tính tổng dãy con, ta thấy cả hai đều có độ phức tạp $O(\log N)$, cải thiện rõ rệt so với hai cách cơ sở.

4.3 Cập nhật giá trị phần tử

Định lý. Giả sử khi thay đổi $a[k]$, các phần tử của mảng C bị ảnh hưởng là

$$C[p_1], C[p_2], \dots, C[p_m].$$

Khi đó

$$p_1 = k, \quad p_{i+1} = p_i + 2^{l_i},$$

trong đó l_i là số chữ số 0 liên tiếp ở cuối biểu diễn nhị phân của p_i . Nói cách khác, $2^{l_i} = \text{lowbit}(p_i)$.

Từ định lý này, ta có cách cập nhật phần tử: nếu cộng x vào $a[k]$, thì các vị trí $C[p_1], C[p_2], \dots, C[p_m]$, với $p_m \leq n < p_{m+1}$, cũng phải được cộng x .

```
procedure ADD(k, x)
begin
  while k <= n do
    begin
      C[k] <- C[k] + x
      k <- k + LOWBIT(k)
    end
  end
end
```

Ví dụ, với dãy $a[1], \dots, a[9]$, nếu cộng x vào $a[3]$, ta có

$$p_1 = 3, \quad p_2 = 3 + 2^0 = 4, \quad p_3 = 4 + 2^2 = 8, \quad p_4 = 8 + 2^3 = 16 > 9.$$

Vì vậy cần cộng x vào $C[3], C[4]$ và $C[8]$.

Bổ đề. Nếu dãy vị trí bị $a[k]$ ảnh hưởng là

$$C[p_1], C[p_2], \dots, C[p_m], \quad p_1 < p_2 < \dots < p_m,$$

và l_i là số chữ số 0 liên tiếp ở cuối biểu diễn nhị phân của p_i , thì

$$l_1 < l_2 < \dots < l_m.$$

Chứng minh. Vì $C[p_i]$ chứa $a[k]$, nên

$$p_i - 2^{l_i} + 1 \leq k \leq p_i.$$

Giả sử tồn tại i sao cho $l_i \geq l_{i+1}$. Từ việc $C[p_{i+1}]$ cũng chứa $a[k]$, ta có

$$p_{i+1} - 2^{l_{i+1}} + 1 \leq k \leq p_i,$$

suy ra

$$p_{i+1} \leq p_i + 2^{l_{i+1}} - 1. \quad (1)$$

Mặt khác, do $p_i < p_{i+1}$ và $l_i \geq l_{i+1}$, vị trí tiếp theo có thể có ít nhất l_{i+1} chữ số 0 ở cuối không thể xuất hiện trước

$$p_i + 2^{l_{i+1}}.$$

Vì thế

$$p_{i+1} \geq p_i + 2^{l_{i+1}}. \quad (2)$$

Hai bất đẳng thức (1) và (2) mâu thuẫn. Do đó $l_1 < l_2 < \dots < l_m$.

Chứng minh định lý. Vì $p_1 < p_2 < \dots < p_m$ và mọi $C[p_i]$ đều chứa $a[k]$, vị trí nhỏ nhất chắc chắn là $p_1 = k$. Trong dãy p , vị trí

$$p_i + 2^{l_i}$$

là vị trí nhỏ nhất sau p_i có số chữ số 0 ở cuối lớn hơn l_i . Nếu có một $p_i + x$ nhỏ hơn vị trí này, thì $x < 2^{l_i}$, số chữ số nhị phân cần xét của x không đủ để tạo ra nhiều chữ số 0 ở cuối hơn, mâu thuẫn với bổ đề.

Đặt $p_{i+1} = p_i + 2^{l_i}$. Khi đó p_{i+1} có ít nhất $l_i + 1$ chữ số 0 ở cuối. Từ

$$p_i - 2^{l_i} + 1 \leq k \leq p_i$$

suy ra

$$p_{i+1} - 2^{l_{i+1}} + 1 \leq p_i + 2^{l_i} - 2 \cdot 2^{l_i} + 1 = p_i - 2^{l_i} + 1 \leq k \leq p_i \leq p_{i+1}.$$

Vì thế $C[p_{i+1}]$ cũng chứa $a[k]$, và quan hệ truy hồi cần chứng minh là

$$p_{i+1} = p_i + 2^{l_i}.$$

4.4 Tính tổng dãy con

Tổng một dãy con có thể chuyển thành hiệu của hai tổng tiền tố, nên chỉ cần biết cách tính

$$S(k) = a[1] + a[2] + \dots + a[k].$$

Theo định nghĩa

$$C[k] = a[k - 2^l + 1] + \dots + a[k],$$

với l là số chữ số 0 liên tiếp ở cuối biểu diễn nhị phân của k . Bắt đầu từ $k_1 = k$, ta dùng công thức

$$k_{i+1} = k_i - 2^{l_i},$$

trong đó l_i là số chữ số 0 liên tiếp ở cuối biểu diễn nhị phân của k_i . Tiếp tục cho đến khi $k_{m+1} = 0$, ta được

$$S(k) = C[k_1] + C[k_2] + \dots + C[k_m].$$

```

function SUM(k)
begin
  ans <- 0
  while k > 0 do
    begin
      ans <- ans + C[k]
      k <- k - LOWBIT(k)
    end
  end
  return ans
end

```

Ví dụ, để tính

$$a[1] + a[2] + a[3] + a[4] + a[5] + a[6] + a[7],$$

ta có

$$k_1 = 7, \quad k_2 = 7 - 2^0 = 6, \quad k_3 = 6 - 2^1 = 4, \quad k_4 = 4 - 2^2 = 0.$$

Do đó

$$a[1] + a[2] + a[3] + a[4] + a[5] + a[6] + a[7] = C[7] + C[6] + C[4].$$

4.5 Tổng ma trận con

Mảng cây cũng có thể mở rộng sang hai chiều. Khi đó ta chỉ cần tính tổng của hình chữ nhật bắt đầu từ $(1, 1)$ đến (x, y) , rồi dùng cộng trừ để lấy tổng của một ma trận con bất kỳ.

Gọi $S(x, y)$ là tổng hình chữ nhật $[1, x] \times [1, y]$. Khi cần tổng của $[x_1, x_2] \times [y_1, y_2]$, ta dùng:

$$\begin{aligned} \text{sum}(x_1, y_1, x_2, y_2) &= S(x_2, y_2) - S(x_1 - 1, y_2) \\ &\quad - S(x_2, y_1 - 1) + S(x_1 - 1, y_1 - 1). \end{aligned}$$

Cập nhật và truy vấn hai chiều có dạng lồng hai vòng lặp của phiên bản một chiều:

```

procedure ADD2D(x, y, delta)
begin
  i <- x
  while i <= n do
    begin
      j <- y
      while j <= m do
        begin
          C[i][j] <- C[i][j] + delta
          j <- j + LOWBIT(j)
        end
      end
      i <- i + LOWBIT(i)
    end
  end
end

```

```

function SUM2D(x, y)
begin
  ans <- 0
  i <- x

```



```

while i > 0 do
begin
  j <- y
  while j > 0 do
  begin
    ans <- ans + C[i][j]
    j <- j - LOWBIT(j)
  end
  i <- i - LOWBIT(i)
end
return ans
end

```

Mỗi thao tác trong hai chiều có độ phức tạp $O(\log n \log m)$.

5 Kết luận

Qua các ví dụ trên, ta thấy khi dùng cấu trúc cây để xử lý bài toán, phạm vi tìm kiếm thường được thu hẹp đáng kể. Trong mỗi lần truy vấn, thuật toán có thể chọn đúng một đường đi bắt đầu từ gốc, tránh được các phép tìm kiếm không cần thiết.

Vì thông thường ta chỉ đi theo một đường từ gốc xuống, tốc độ duyệt cây phụ thuộc rất nhiều vào độ sâu của cây. Do đó, làm thế nào để giảm chiều cao của cây trở thành điểm then chốt của thuật toán.

So với cây theo nghĩa truyền thống, mảng cây nhìn bề ngoài không hẳn là một cây. Tuy vậy, tư tưởng cốt lõi của nó vẫn cùng một bản chất với cấu trúc cây: tổ chức thông tin theo từng mức để thu hẹp phạm vi cập nhật và truy vấn.